MEK3100 – Advanced Python Programming for Engineers

Programming Project

Fall 2026

Due: Sunday, 18 October 2026 23:59

Guidelines and warnings:

- Use your own approach and algorithm to write the code.
- Group members should discuss the problem together to come up with a solution (an algorithm) before writing the actual code.
- Any copies from the internet or other groups will give you an automatic **FAIL** for the project for every involved party (the giver and the taker).
- **Each group** must submit **one Python file** (e.g., **project.py**).
- The submission will be done electronically to Canvas.

Project description

Bank Account Management System

The aim of this project is to create a bank account management system using classes and decorators. You have to implement a class called **BankAccount** that represents a bank account. The **BankAccount** class should have the following functionalities:

- **Account Creation**: Create a new bank account with an initial balance
- **Deposit**: Deposit money into the account
- **Withdrawal**: Withdraw money from the account
- **Balance Inquiry**: Check the current balance of the account
- **Transaction History**: Keep track of all the transactions made on the account
- **Retrieve Transactions**: Retrieve the transaction history for an account
- **Account Statement Generation and Print**: Generate and print an account statement that includes transaction details and the current balance
- **Account Summary Generation and Print**: Generate and print an account summary that includes account details including the account number, current balance, and the number of transactions
- **Account Closure**: Close the account, setting the balance to zero and clearing the transaction history
- **Check if Account is Closed**: Implement a method (e.g. **is_account_closed**) to check if the account is closed

Requirements:

- Implement the BankAccount class with the specified functionalities.
- Use a decorator to handle withdrawal operations and ensure sufficient balance in the account.
- Use a decorator to measure the execution time of deposit and withdrawal operations.
- You can use a list to store the transaction history. Each transaction can be represented as a dictionary containing information such as the transaction type (deposit or withdrawal), the amount, and the timestamp. For the *timestamp*, use ***strftime*** function in the ***time*** module.
- Provide a menu-driven program with all the above functionalities.

Lykke til 😊